



GIA - IAA

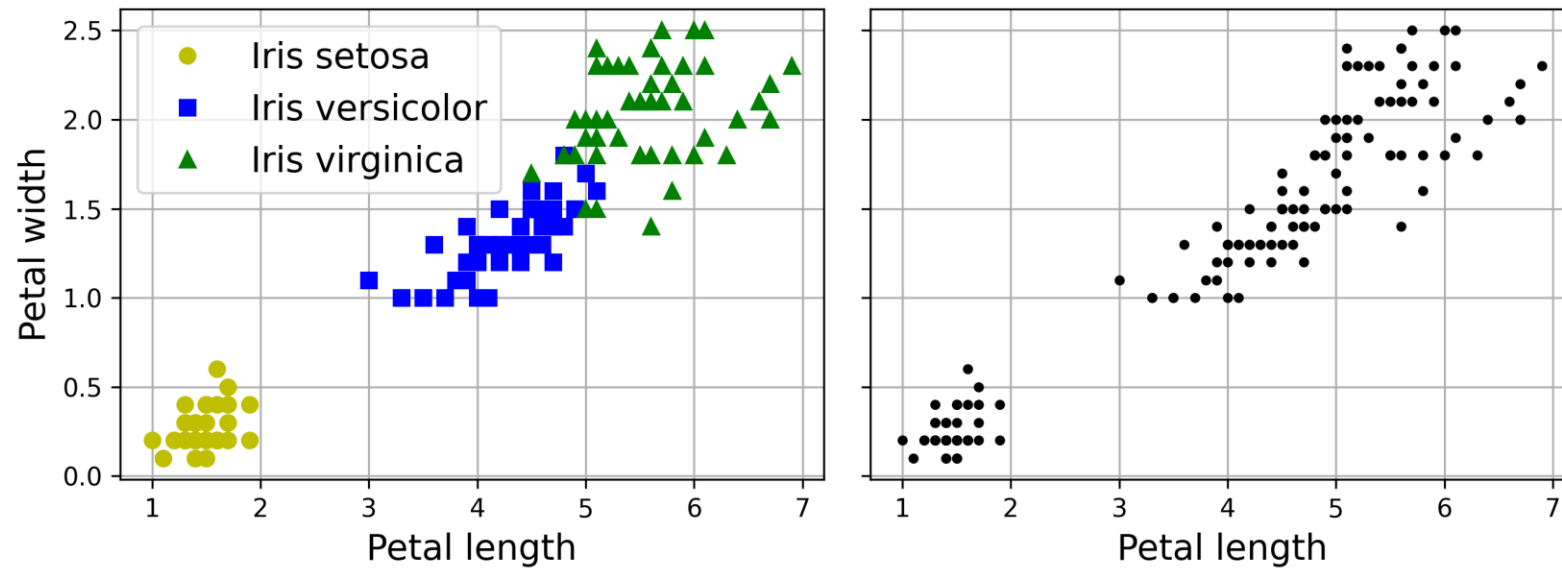
Introducció a

l'Aprenentatge Automàtic

8. Lab: Clustering

Jordi Luque
jordi.luque.serrano@upc.edu

Clustering vs Classification



Clustering

❖ There are 4 major categories of clustering algorithms:

1. Centroid-based Clustering
2. Density-based Clustering
3. Distribution-based Clustering
4. Hierarchical Clustering

DBSCAN



k-means



K-Means Clustering

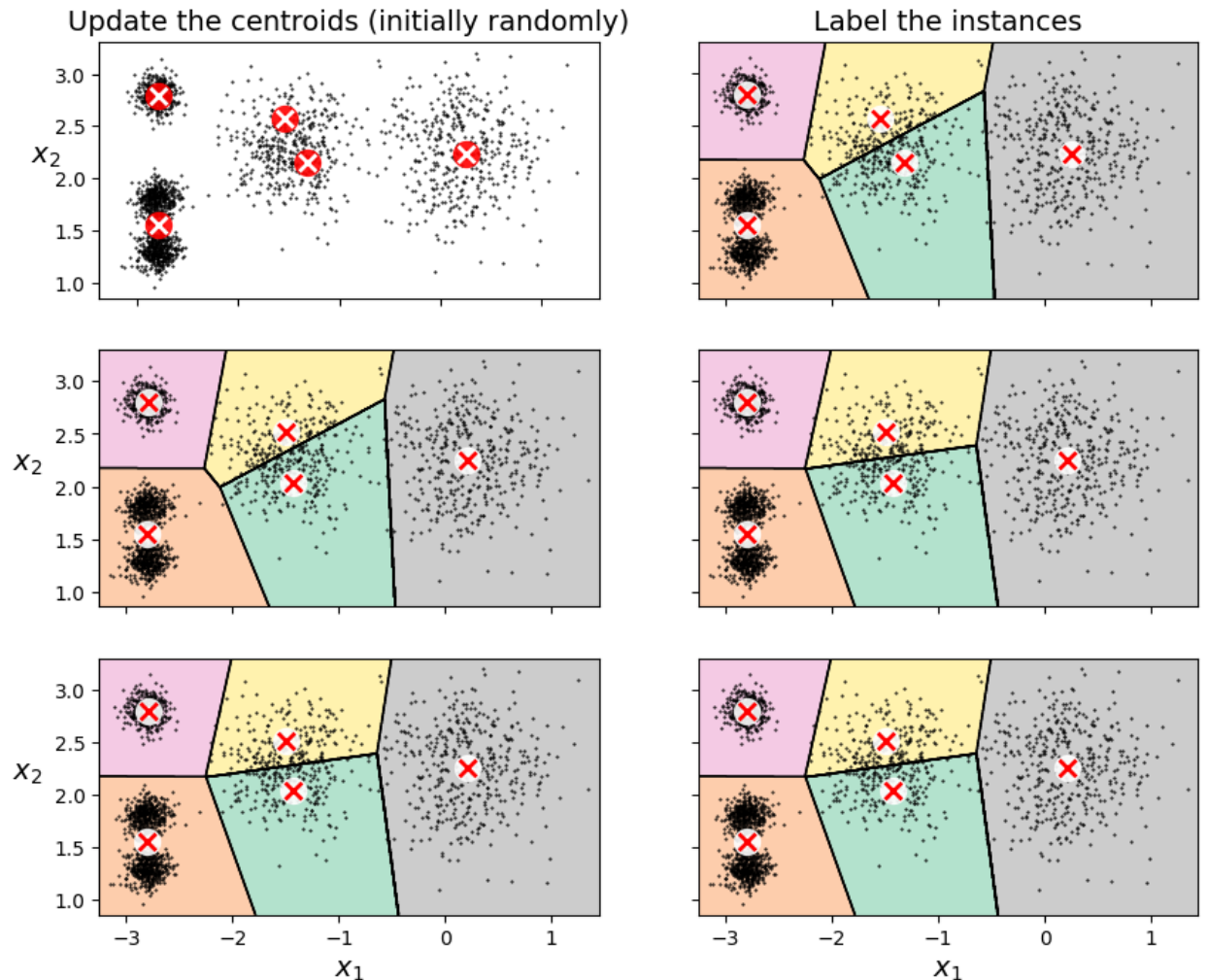
- ❖ For many of us, if we're familiar with clustering algorithms at all, we know about k-means clustering. Is one of the most widely used clustering algorithms. It aims to partition the data into K clusters, **where K is a predefined number**.

PLAYTIME: Cluster these points: [1,1], [1,2], [1,5], [3,2], [3,4], [4,1], [4,4], [5,3], [5,5], [6,2], [6,6], [7,7]
Which are the centroids? Print the label for each data point.

How you decided on the number k of clusters?

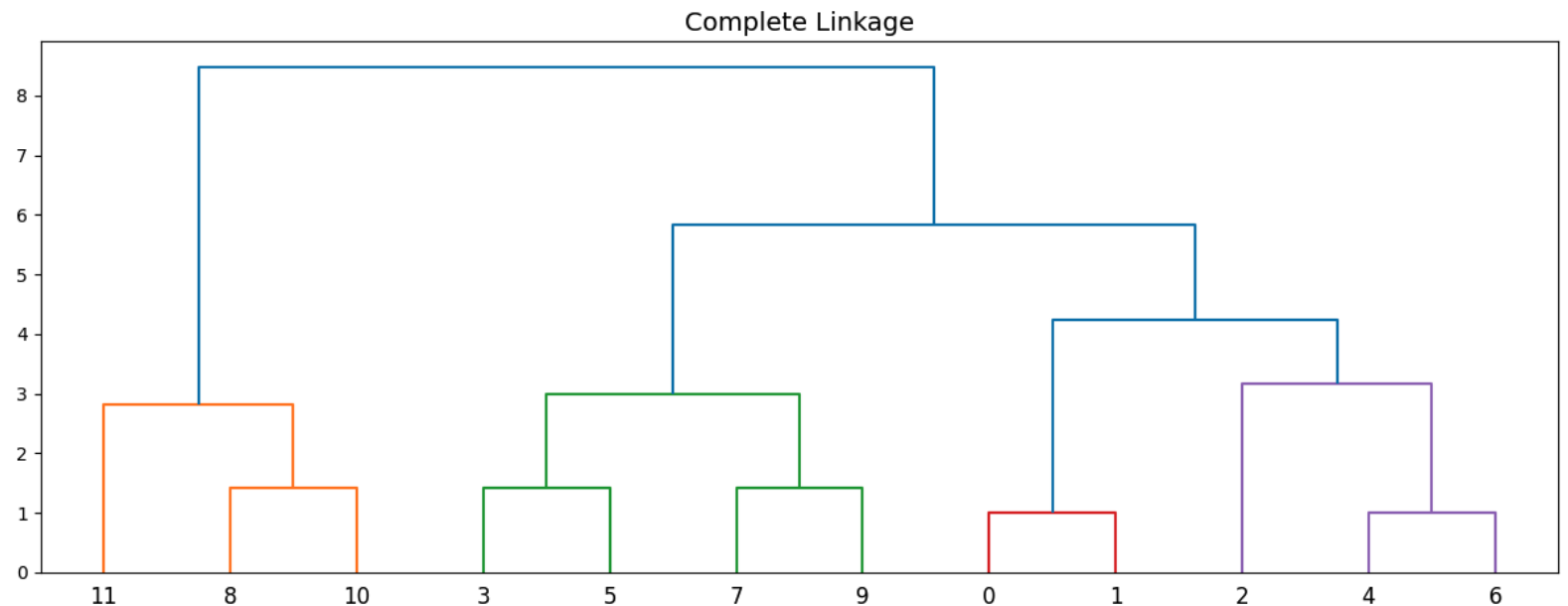
K-Means Clustering

- ❖ It iteratively assigns data points to clusters based on the proximity to the cluster centroids and updates the centroids until convergence.
- ❖ One of the challenges with k-means is that you need to specify a number of clusters ("k") in order to use it.



Hierarchical Clustering

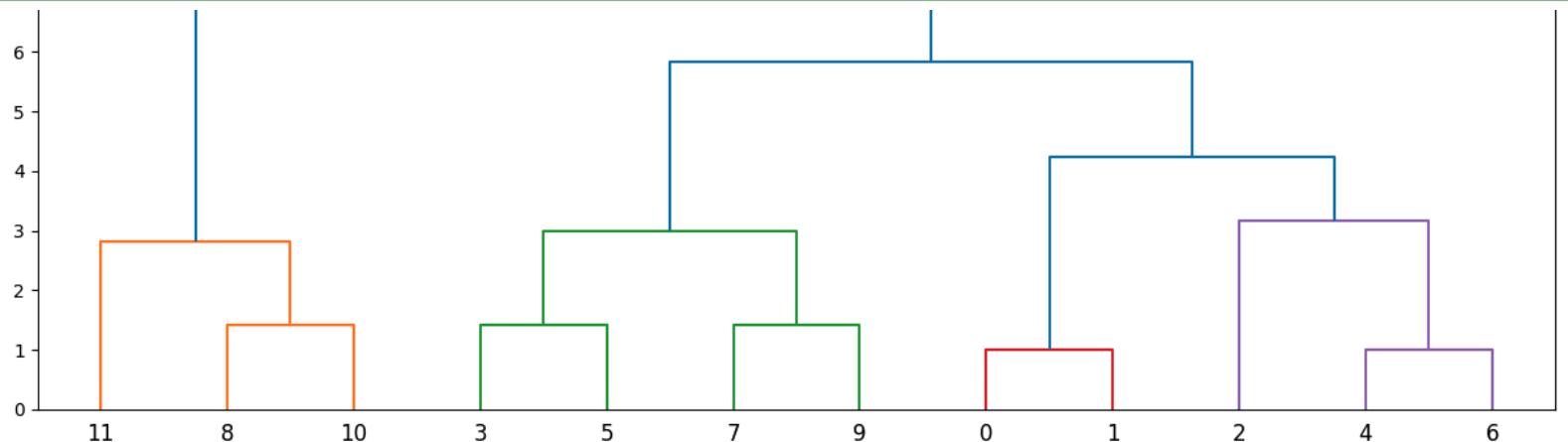
- ❖ This approach creates a hierarchical decomposition of the data by iteratively merging or splitting clusters. It can be agglomerative (bottom-up) or divisive (top-down) in nature. The result is often visualized as a **dendrogram**, which shows the nested clusters at different levels of similarity.



Hierarchical Clustering

- ❖ This approach creates a hierarchical decomposition of the data by iteratively merging or splitting clusters. It can be agglomerative (bottom-up) or divisive (top-down) in nature. The result is often visualized as a **dendrogram**, which shows the nested clusters at different levels of similarity.

PLAYTIME: Check the hierarchy model documentation and plot dendrograms for the different linkage methods available. How clusters are created? Which are bottom-up and top-down?



<https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.linkage.html>

NCI60 Dataset

RDocumentation

made4 (version 1.46.0)

NCI60: Microarray gene expression profiles of the NCI 60 cell lines

Description

NCI60 is a dataset of gene expression profiles of 60 National Cancer Institute (NCI) cell lines. These 60 human tumour cell lines are derived from patients with leukaemia, melanoma, along with, lung, colon, central nervous system, ovarian, renal, breast and prostate cancers. This panel of cell lines have been subjected to several different DNA microarray studies using both Affymetrix and spotted cDNA array technology. This dataset contains subsets from one cDNA spotted (Ross et al., 2000) and one Affymetrix (Staunton et al., 2001) study, and are pre-processed as described by Culhane et al., 2003.

Usage

```
data(NCI60)
```

<https://rdocumentation.org/packages/made4/versions/1.46.0/topics/NCI60>

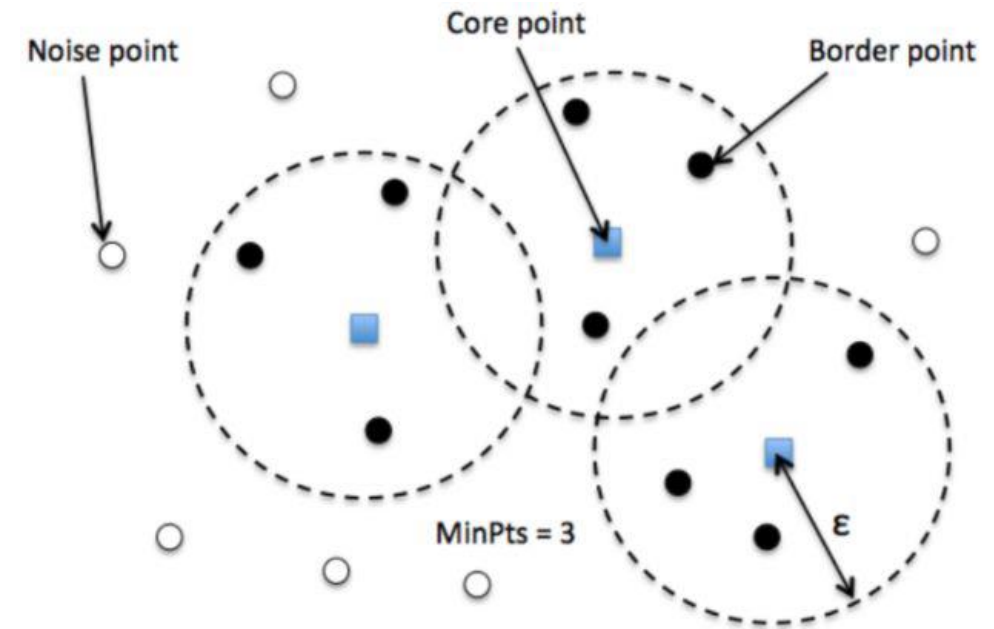
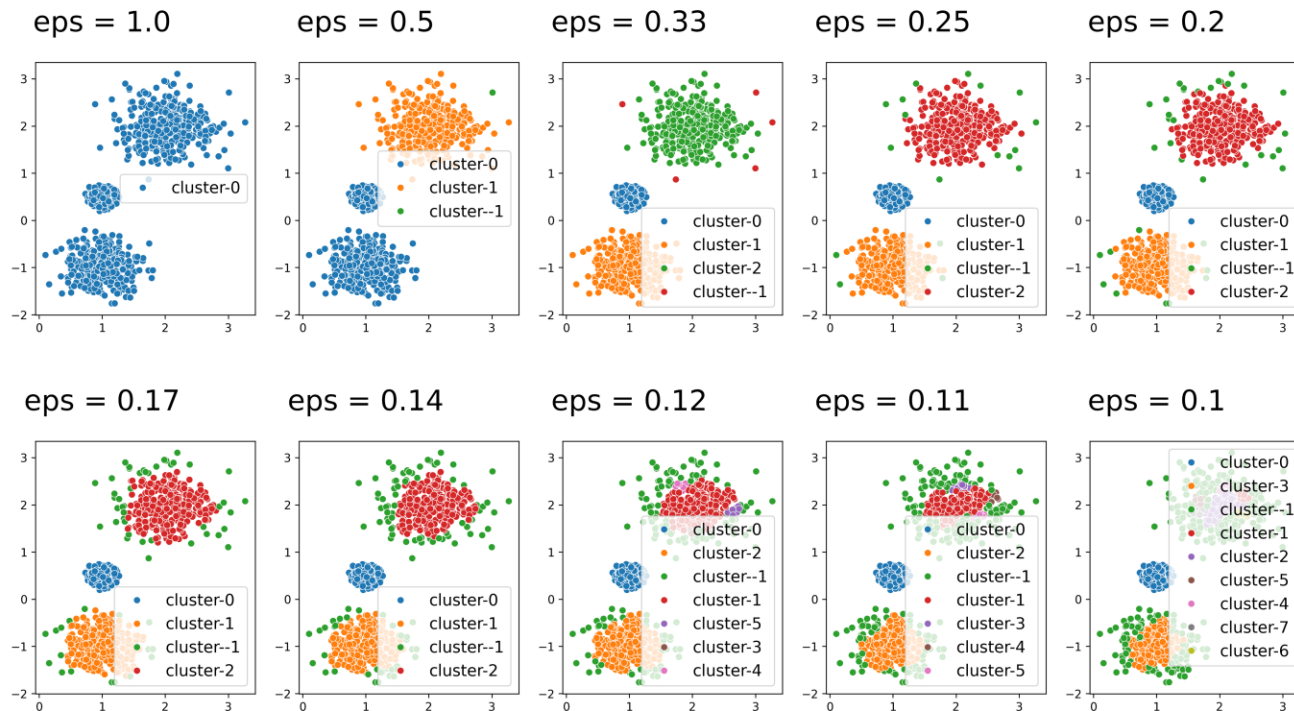
DBSCAN Clustering

- ❖ DBSCAN (Density-Based Spatial Clustering of Applications with Noise) group together data points that are within a dense region in the data space and separate them from sparser regions. Only have two parameters.
- ❖ Radius (epsilon)
 - High: Larger clusters. Irregular.
 - Small: Smaller clusters, more noise
- points. Spherical
- ❖ Num. min. points:
 - High: Denser clusters
 - Smaller: Sparser clusters

Algorithm 1: DBSCAN Algorithm

```
Input : points, minSamples
Output: cluster, type
for point in points do
    if neighbors[point].count()  $\geq$  minSamples then
        corePoints.push(point)
        type[point] = 'core point'
    end
end
for cp in corePoints do
    if cluster[cp] is NULL then
        ++clusterCount
        cluster[cp] = clusterCount
        queue = []
        while not queue.empty() do
            for neigh in neighbors[cp] do
                cluster[neigh] = clusterCount
                if type[neigh] == 'core point' then
                    queue.push(neigh)
                end
            else
                type[neigh] = 'non-core point'
            end
            end
        end
    end
end
for point in points do
    if type[point] is NULL then
        type[point] = 'outlier'
    end
end
```

DBSCAN Clustering



PLAYTIME: Fix Epsilon (Radius) and plot a similar graph for the different values of min_samples. How the creation of cluster is affected?

Olivetti Faces Dataset

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More ▾](#)

[Prev](#) [Up](#) [Next](#)

scikit-learn 1.3.2
[Other versions](#)

Please [cite us](#) if you use the software.

[sklearn.datasets.fetch_olivetti_faces](#)
[fetch_olivetti_faces](#)
Examples using
[sklearn.datasets.fetch_olivetti_faces](#)

sklearn.datasets.fetch_olivetti_faces

```
sklearn.datasets.fetch_olivetti_faces(*, data_home=None, shuffle=False, random_state=0, download_if_missing=True, return_X_y=False)
```

[\[source\]](#)

Load the Olivetti faces data-set from AT&T (classification).

Download it if necessary.

Classes	40
Samples total	400
Dimensionality	4096
Features	real, between 0 and 1

Read more in the [User Guide](#).

Parameters:

- data_home** : *str or path-like, default=None*
Specify another download and cache folder for the datasets. By default all scikit-learn data is stored in '~/scikit_learn_data' subfolders.
- shuffle** : *bool, default=False*
If True the order of the dataset is shuffled to avoid having images of the same person grouped.
- random_state** : *int, RandomState instance or None, default=0*
Determines random number generation for dataset shuffling. Pass an int for reproducible output across multiple function calls. See [Glossary](#).

Olivetti Faces Dataset

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More ▾](#)

[Prev](#) [Up](#) [Next](#)

scikit-learn 1.3.2
[Other versions](#)

sklearn.datasets.fetch_olivetti_faces

```
sklearn.datasets.fetch_olivetti_faces(*, data_home=None, shuffle=False, random_state=0)
```

PLAYTIME: The classic Olivetti faces dataset contains 400 grayscale 64 × 64-pixel images of faces. Each image is flattened to a 1D vector of size 4,096. 40 different people were photographed (10 times each), and the usual task is to train a model that can predict which person is represented in each picture. Load the dataset using the `sklearn.datasets.fetch_olivetti_faces()` function

```
sklearn.datasets.fetch_olivetti_faces()
```

Classes	40
Samples total	400
Dimensionality	4096
Features	real, between 0 and 1

Read more in the [User Guide](#).

Parameters:

- data_home** : *str or path-like, default=None*
Specify another download and cache folder for the datasets. By default all scikit-learn data is stored in `'~/scikit_learn_data'` subfolders.
- shuffle** : *bool, default=False*

PLAYTIME: Next, cluster the images using K-Means, and ensure that you have a good number of clusters. Plot the silhouette score against the number of clusters k . Plot the inertia score against the number of clusters k . Which value of k do you pick? why?

Olivetti Faces Dataset

